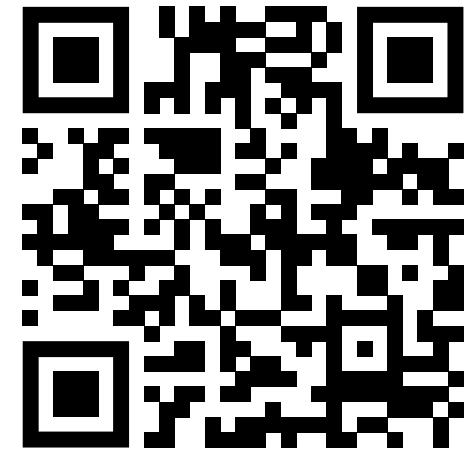


Programmierung in C++

Standard-Container

Prof. Dr. Bernd Lüdemann-Ravit

Ansprechpartner: Dan Eisenkrämer



<https://poll.hs-kempten.de/poll/>



Einführung

Lernziele

- Das Konzept von Containern wurde verstanden
 - Verschiedene Container-Arten sind bekannt
- Die Anforderungen an Standard-Container sind bekannt
 - Der Unterschied zu „Nahezu Containern“ ist klar

Einführung

Motivation



- Container sind Datenstrukturen zur **Verwaltung und Organisation von Datensammlungen**
- z.B. das **C-Style-Array** ermöglicht beliebige Elemente eines Datentyps zu verwalten:
 - Speichert jedoch nicht, wie viele Elemente verwaltet werden
 - Lässt sich in der Größe nicht anpassen
 - Bietet keine Funktionalitäten, um ein bestimmtes Element einfach zu finden
 - Gibt reservierten Speicher nicht automatisch wieder frei
- z.B. unsere **„Vector“-Klasse**, in der wir Double-Werte verwalten
 - Gibt reservierten Speicher nach dem RAII-Prinzip automatisch wieder frei
 - Anzahl der verwalteten Elemente lässt sich anpassen
 - Kann mit Funktionalitäten erweitert werden
 - Kann (erstmal) nur Double-Werte verwalten und keine anderen Datentypen

Einführung

Motivation



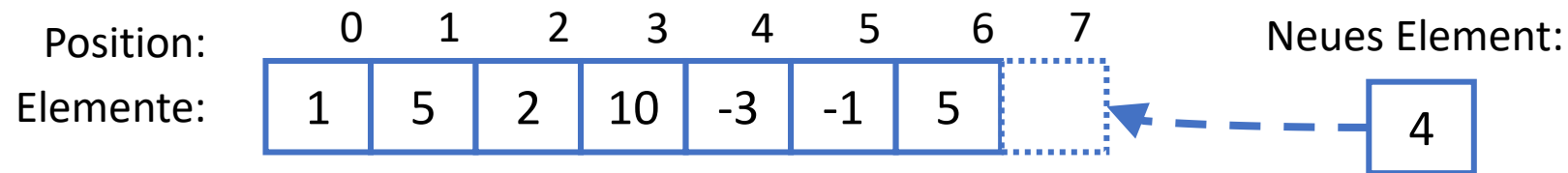
- „A **Container** is an object used to store other objects and taking care of the management of the memory used by the objects it contains.”
(Definition CppReference)
- Container-Klassen der Standard-Bibliothek sind auf **einfache Bedienung, Sicherheit** und **Effizienz** optimierte Container
 - Kapseln **Speicherverwaltung** und **Algorithmen** (z.B. zum Einfügen, Löschen, Finden, Sortieren)
 - Sind in ihrer Implementierung für **Performance** optimiert
 - Sind **generalisiert**, d.h. sie können für beliebige Datentypen/Objekte verwendet werden
 - Setzen das **RAII-Prinzip** um
 - Durch einen internen „Allokator“ und „Deallokator“ wird Speicher korrekt reserviert und wieder freigegeben
 - Reduzieren Fehler durch manuelle Speicherverwaltung

Einführung

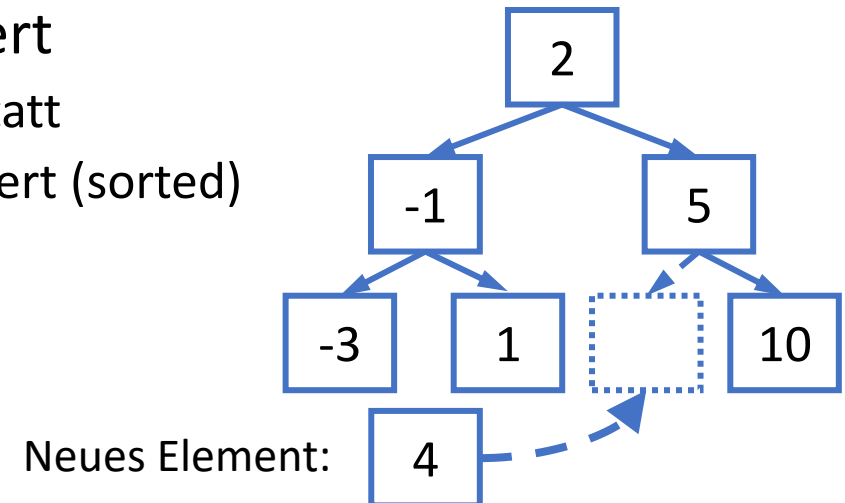


Sequenzielle vs. assoziative Container

- **Sequenzielle Container** speichern die Objekte ungeordnet in der Reihenfolge, die der Benutzer direkt durch sein Programm vorgibt
 - Der Zugriff auf die Elemente findet über die Position des Elements in der Reihenfolge statt



- In **assoziativen Container** werden die Objekte mit einem oder auch ihrem eigenen Wert (dem Schlüssel des Elements) assoziiert
 - Der Zugriff auf die Elemente findet über den Schlüssel statt
 - Unterscheidung zwischen: Unsortiert (hashed) und sortiert (sorted)
 - Können nur aus einem Schlüssel bestehen oder aus einem Key-Value-Pair
 - Elemente können einzigartig (unique) sein oder mehrfach vorkommen (multiple)



Einführung

Übersicht über ausgewählte Standard-Container



Sequenzielle Container		Assoziative Container	
C++11	<u><code>std::array<T, N></code></u>		Array mit fester Größe von N Elementen des Typs T
	<code>std::deque<T></code>		Warteschlange mit Verweis auf das erste und letzte Element vom Typ T
	<u><code>std::list<T></code></u>		doppelt verkettete Liste von N Elementen des Typs T
	<code>std::forward_list<T></code>		einfach verkettete Liste von N Elementen des Typs T
	<u><code>std::vector<T></code></u>		dynamisches Array mit Elementen des Typs T
C++26			dynamisches Array mit Elementen des Typs T und maximaler Größe N
	<code>std::inplace_vector<T,N></code>		
		<u><code>std::map<K, V></code></u>	sortierte Menge von einzigartigen Schlüsseln vom Typ K und zugehörigen Wertobjekten des Typs V
		<code>std::multimap<K,V></code>	sortierte Menge mit Schlüsseln K und zugehörigen Werten V ; Schlüssel können mehrfach vorkommen
		<u><code>std::set<K></code></u>	sortierte Menge von einzigartigen Schlüsseln K
		<code>std::multiset<K></code>	sortierte Menge von Schlüsseln K , die mehrfach vorkommen dürfen
		<u><code>std::unordered_map<K,V></code></u>	unsortierte <code>std::map</code> mithilfe einer Hash-Funktion
		<code>std::unordered_multimap<K,V></code>	unsortierte <code>std::multimap</code> mithilfe einer Hash-Funktion
		<code>std::unordered_set<K></code>	unsortiertes <code>std::set</code> mithilfe einer Hash-Funktion
		<code>std::unordered_multiset<K></code>	unsortiertes <code>std::multiset</code> mithilfe einer Hash-Funktion

- Zusätzlich existieren Container-Adapter („Wrapper“), die eine besondere Schnittstelle für sequenzielle Container zur Verfügung stellen:
 - **`std::stack`** (LIFO-Struktur) und **`std::queue`** (FIFO-Struktur)
 - **C++23** **`std::flat_set`** und **`std::flat_map`** verwalten die Elemente intern in sequenziellen Containern

Einführung

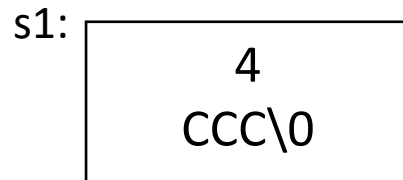
Standard Strings



```
#include <string>
```

- Strings vereinfachen das Arbeiten mit Zeichenketten
 - Gelten als **Pseudo-Container**, da sie nur Zeichen speichern und keine anderen Datentypen
 - Besitzen viele Interaktionsmöglichkeiten mit C-Style Char Arrays
 - Sind stark optimiert auf Speicherplatz und Laufzeit
 - Tatsächliche Performanz sehr abhängig von der Laufzeit-Umgebung
 - Stellen viele Operationen zur Manipulation der Zeichenkette zur Verfügung
- Kurze Strings werden im String-Objekt selbst gespeichert
 - Erst für längere Strings wird dynamischer Speicherplatz verwendet
 - „Kurz“ ist abhängig vom Compiler (Faustregel: 14 Zeichen)

```
std::string s1{"CCC"}; // short string
```



```
std::string s2{"Campodunum Chaos Coder"}; // long string
```



https://en.cppreference.com/w/cpp/string/basic_string

Einführung

Standard Strings

- Der + Operator konkateniert zwei Strings
 - Auch mit einem C-style String oder Character
- Zuweisungsoperatoren und Indizierung sind definiert
 - Indizierung via [], at() oder Iteratoren
- Vergleichsoperatoren für std::string untereinander oder mit C-style Strings oder Literalen
- Umwandlung in C-style String mit .c_str()
- Suffix s zur impliziten Definition eines Strings (*std::literals::string_literals*)



```
std::string s{"Hallo Welt"};
s = s + '\n'; // Zeilenumbruch anhängen
```

```
#include <string>
```

```
std::string n = "Prof. Lüdemann";
n += "-Ravit";           // "-Ravit" wird angehängt
std::string f = n.substr(6,8); // f="Lüdemann"
n.replace(0,5,"bernd");   // n="bernd Lüdemann-Ravit"
if (!n.contains("Bernd")) C++23
    n[0] = std::toupper(n[0]);
// n="Bernd Lüdemann-Ravit"
```

```
void print(const std::string& s) {
    printf("Ich mag printf: %s\n", s.c_str());
    std::cout << "Ich nicht: " << s << std::endl;
}
```

```
using namespace std::literals::string_literals;
int main()
{
    auto katze = "Katze"s;    // std::string
    auto hund = "Hund";       // const char*
}
```

Einführung

Basic String



```
#include <string>
```

- `std::string` ist ein Alias für die Klasse ***basic_string*** mit Elementen vom Typ *char*

```
namespace std {  
    using string = basic_string<char>;  
}
```

- Basic String kann nicht nur Zeichen vom Typ *char* verwalten, sondern alle möglichen Zeichen
 - Zum Beispiel vom Breitzichen-Typ *wchar_t* (meistens 2 oder 4 Bytes groß)
 - Angenommen, es gäbe einen Zeichentyp für japanische Zeichen *Jchar*, könnte ein entsprechender *Jstring* definiert werden

```
using Jstring = std::basic_string<Jchar>;
```

Einführung

Pair



```
#include <utility>
```

- `std::pair` speichert zwei Elemente mit unterschiedlichen Datentypen
 - Für assoziative Container mit **Key-Value-Paaren** wird `std::pair` intern verwendet
 - `std::make_pair` erzeugt ein Paar automatisch aus der Parameterliste
 - Auf das erste Element kann mit `.first` zugegriffen werden
 - Auf das zweite Element kann mit `.second` zugegriffen werden

```
Koordinaten:  
x: 10.5, y: 20.8
```

```
// Erstelle ein Paar mit einem Namen und einem Alter  
std::pair<std::string, int> person{"Alice", 30};  
std::cout << "Name: " << person.first << '\n';  
std::cout << "Alter: " << person.second << '\n';  
  
// Ändern der Werte im Paar  
person.first = "Bob";  
person.second = 25;  
std::cout << "Name: " << person.first << '\n';  
std::cout << "Alter: " << person.second << '\n';  
  
// Erstellung eines neuen Paares mit make_pair  
auto coordinates = std::make_pair(10.5, 20.8);  
std::cout << "\nKoordinaten:\n";  
std::cout << "x: " << coordinates.first  
            << ", y: " << coordinates.second << '\n';
```

```
Name: Alice  
Alter: 30  
Name: Bob  
Alter: 25
```

Einführung

C++11 Tupel



```
#include <tuple>
```

- **std::tuple** generalisiert `std::pair` und erlaubt Tupel mit null und mehr Elementen mit beliebigen Datentypen
 - **std::make_tuple** erzeugt ein Tupel automatisch aus der Parameterliste
 - Zugriff auf die Elemente des Tupels geschieht über `std::get<x>(tuple)`
 - **x** ist der Index des Elements oder ein im Tupel einzigartiger Datentyp

```
std::tuple t0{}; // Leeres Tupel
std::tuple<std::string, int, double> t1{ "Hai", 123, 3.14 };
auto t2 = std::make_tuple(std::string{ "Fisch" }, 10, 1.23);
std::tuple t3{ "Made"s, 20, 9.99 };
std::get<1>(t2) = 100;
std::cout << "Tier: " << std::get<0>(t1)
          << "\nPreis: " << std::get<double>(t1);
```

```
Tier: Hai
Preis: 3.14
```

Einführung

Zusammenfassung



- Container beinhalten eine Reihe von Objekten und bieten Funktionalitäten zum Einfügen, Löschen, Finden, Sortieren, ... dieser
 - Container verwenden intern einen „Allokator“, um Speicher zu reservieren und freizugeben. Standardmäßig wird dazu `new` und `delete` verwendet.
 - „Nahezu Container“ bieten viele, aber nicht alle notwendigen Funktionalitäten für einen Standardcontainer. Dazu gehören z.B. das C-Style Array (`int[5]`) und der Standard String (`std::string`)
- **Sequenzielle Container** speichern die Objekte ungeordnet in der Reihenfolge, die der Benutzer direkt durch sein Programm vorgibt
 - Der Zugriff auf die Elemente findet über die Position des Elements in der Reihenfolge statt
- In **assoziativen Container** werden die Objekte mit einem oder auch ihrem eigenen Wert (dem Schlüssel des Elements) assoziiert
 - Der Zugriff auf die Elemente findet über den Schlüssel statt
 - Es wird zwischen geordneten und ungeordneten assoziativen Containern unterschieden



Sequenzielle Container

Lernziele

- Die **Funktionsweise** sequenzieller Container der Standard Bibliothek wurde anhand der vorgestellten Beispiele **verstanden**
- Das **Anwendungsgebiet** sowie **Vor- und Nachteile** der sequenziellen Container sind bekannt
- Sequenzielle Container können in eigenen Projekten **verwendet** werden

Sequenzielle Container

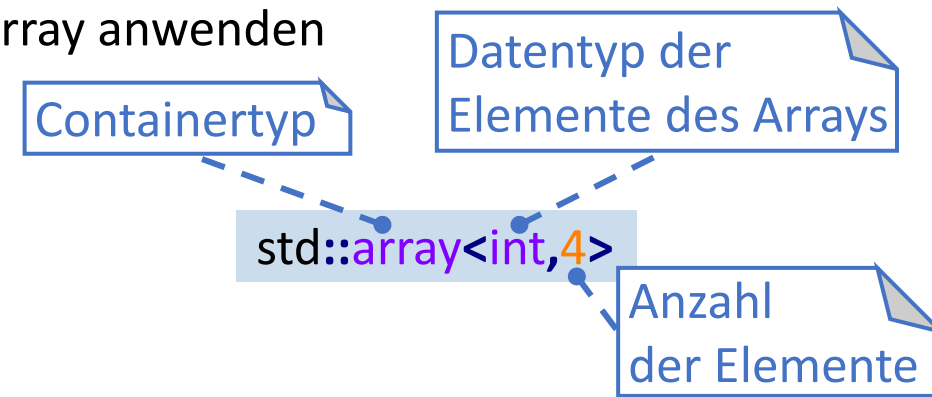
C++11 Standard Array



```
#include <array>
```

- C-Style Arrays führen schnell zu Unannehmlichkeiten und Problemen
 - Die feste Größe, mit der sie angelegt werden, wird nicht automatisch gespeichert
 - Bei Zuweisung eines Kindklassen-Arrays zu einem Pointer der Elternklasse können bei Funktionsaufrufe Offsets entstehen, die zu Laufzeitfehlern führen
- **std::array** ist ein Wrapper für C-Style Arrays zur Lösung obiger Probleme
 - Benötigt bei der Deklaration den Datentyp und die konstante Größe für das Array
 - Wird wie ein C-Style Array initialisiert und verwendet
 - Seine Elemente liegen ebenso auf dem Stack
 - Viele STL-Algorithmen lassen sich einfach auf dem std::array anwenden
 - Kann mit dem = Operator kopiert werden

```
void f(int* p, int sz);           // C-style Interface
int main() {
    std::array<int,4> a = { 1, 2, 3, 4 }; // Initialisierungsliste
    f(a.data(), a.size());           // C-style Verwendung
}
```



<https://en.cppreference.com/w/cpp/container/array>

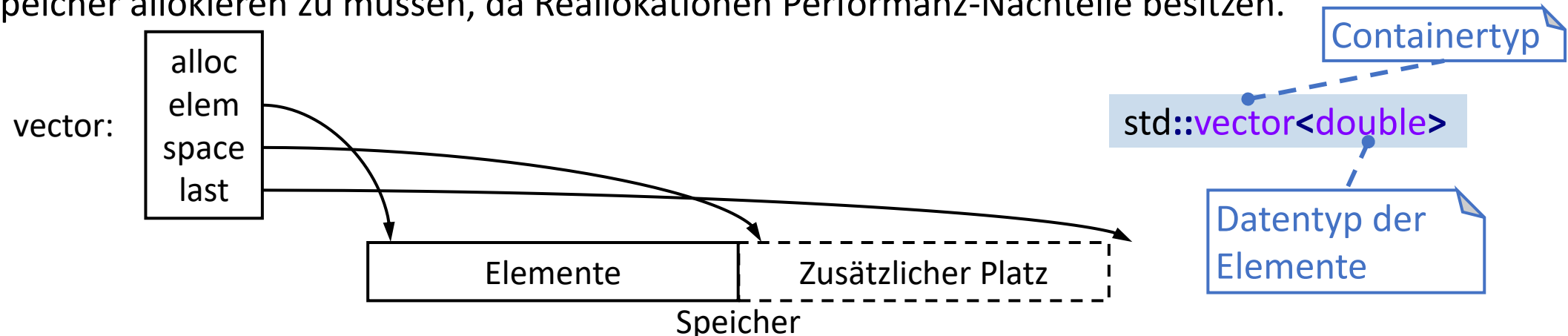
Sequenzielle Container

Standard Vector



```
#include <vector>
```

- Ein **`std::vector`** kann eine Folge von Elementen desselben Datentyps halten, die aufeinanderfolgend im Speicher angelegt sind
 - Die Anzahl der Elemente im Vektor kann auch nach der Initialisierung geändert werden
 - Dazu besitzt der Standard Vector einen Pointer auf das erste Element und einen auf das Element hinter dem letzten
 - Der `std::vector` allokiert dabei automatisch benötigten Speicher und besitzt dafür intern einen entsprechenden Verteiler (alloc)
 - Meistens wird zusätzlicher Speicher allokiert, um nicht bei jedem neuen Element neuen Speicher allokieren zu müssen, da Reallokationen Performanz-Nachteile besitzen.



Sequenzielle Container

Standard Vector



```
#include <vector>
```

- Ein `std::vector` kann eine Folge von Elementen desselben Datentyps halten, die aufeinanderfolgend im Speicher angelegt sind
 - Mit **`size()`** wird die Anzahl der Elemente und mit **`capacity()`** die Größe des allokierten Speichers zurückgegeben
 - `shrink_to_fit()`** gibt den nicht verwendeten Speicher frei und **`reserve()`** setzt ihn auf einen neuen Wert, z.B. wenn die benötigte Größe des `std::vector` bekannt ist.
 - Bei der Initialisierung kann die initiale Größe festgelegt werden oder bereits Elemente hinzugefügt werden

```
std::vector<Entry> phonebook = {  
    {"David Hume",123456},  
    {"Karl Popper",234567},  
    {"Bertrand Russell",345678}  
};
```

```
for (std::size_t i = 0; i < phonebook.size(); ++i)  
    std::cout << phonebook[i] << ' \n';
```

```
David Hume: 123456  
Karl Popper: 234567  
Bertrand Russell: 345678
```

```
std::vector<int> v1 = { 1,2,3,4 }; // Size = 4  
std::vector<std::string> v2; // Size = 0  
std::vector<Gegner*> v3(23); // Size = 23, initialisiert mit nullptr  
std::vector<double> v4(4, 9.9); // Size = 4, initialisiert mit 9.9
```

```
for (const auto& d : v4)  
    std::cout << d << ' \n';
```

```
9.9  
9.9  
9.9  
9.9
```

Die Elemente von `std::vector` befinden sich in einer Reihe => Der Range-Based-For-Loop kann verwendet werden.

Sequenzielle Container

Standard Vector



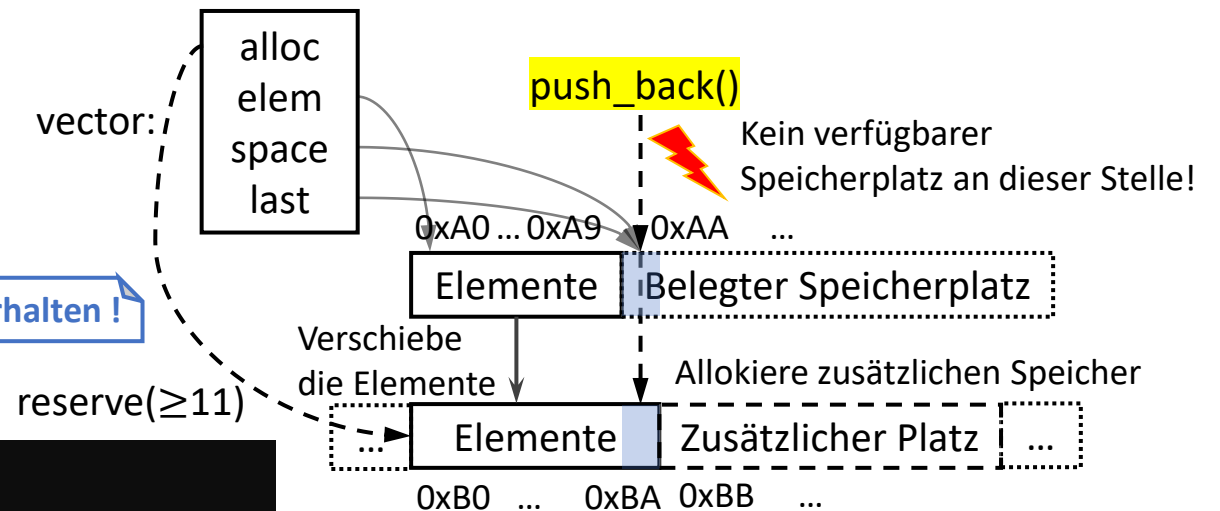
```
#include <vector>
```

- Ein `std::vector` kann eine Folge von Elementen desselben Datentyps halten, die aufeinanderfolgend im Speicher angelegt sind
 - `reserve()`** legt die Größe des Speicherplatzes fest, der von `std::vector` allokiert sein soll
 - Mit **`push_back()`** wird ein Element nach dem letzten Element hinzugefügt
 - Wenn kein Speicherplatz für ein neues Element vorhanden ist, wird mit **`reserve()`** neuer Speicherplatz allokiert und **alle Elemente** an einen neuen, passenden Speicherplatz **verschoben** (Pointer auf die Elemente werden ungültig!)

```
auto* karl = &phonebook[1];
std::cout << "Karl Adresse: " << karl << '\n';

phonebook.push_back({ "Bernd LR", 49152391 });
auto* karlNeu = &phonebook[1];
std::cout << "Karl Adresse (alt): " << karl
<< ", Eintrag: " << *karl << '\n';
std::cout << "Karl Adresse (neu): " << karlNeu
<< ", Eintrag: " << *karlNeu << '\n';
```

```
Karl Adresse: 00000267E7155490
Karl Adresse (alt): 00000267E7155490, Eintrag:
Karl Adresse (neu): 00000267E715DFC0, Eintrag: Karl Popper: 234567
```

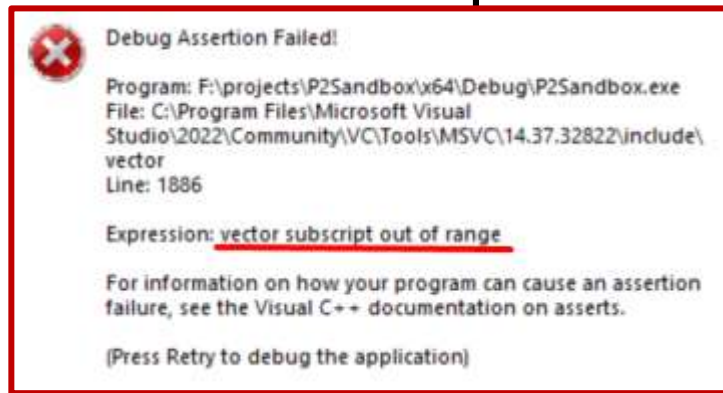


Sequenzielle Container

Range Checking



```
std::vector<Entry> phonebook = { /* ... */ };  
std::cout << phonebook[phonebook.size()];
```



```
std::vector<Entry> phonebook = { /* ... */ };  
std::cout << phonebook.at(phonebook.size());
```

Unbehandelte Ausnahme bei 0x00007FFFE5C3CF19 in P2Sandbox.exe:
Microsoft C++-Ausnahme: std::out_of_range bei Speicherort
0x000000070F84FF3F0.

- Der `std::vector` (so wie die meisten STL- Container) garantiert **keine** sichere Indizierung
 - Es wird nicht überprüft, ob ein Index auf ein Element außerhalb des Containers zugreifen würde => **Bereichsfehler**
 - Der Zugriff auf ein solches Element sorgt für nicht definiertes Verhalten => Es kann zum Absturz führen, aber auch willkürliche Werte zurückgeben
- Die Funktion ***at()*** führt Bereich-Zugriffs-Überprüfungen durch
 - Bei fehlerhaftem Zugriff wird ein „*out_of_range*“-Fehler geworfen, der deterministisches Verhalten gewährleistet.
 - Überprüfungen mit `at()` sind laufzeitaufwändiger als ohne.

Sequenzielle Container



C++11 Initializer List

```
#include <initializer_list>
```

- Häufig möchte man ein Objekt einer eigenen Klasse mit einer flexibel langen Liste von Elementen eines Datentyps initialisieren
 - Hierfür kann man `std::vector` verwenden, was sich anbietet, wenn die Liste auch außerhalb der Initialisierung benötigt wird
 - Benötigt man die Liste nur für die Initialisierung, bietet sich `std::initializer_list` an

```
class Vector {  
public:  
    explicit Vector(std::initializer_list<double>);  
    // Beispiel aus Vorlesung 1 ...  
};
```

Vector.h

```
Vector::Vector(std::initializer_list<double> list)  
    : m_elements{new double[list.size()]},  
      m_size{list.size()}  
{  
    // Kopier die Elemente der Liste in die der Klasse  
    std::copy(list.begin(), list.end(), m_elements);  
}
```

Vector.cpp

```
Vector v1{1, 2, 3, 4, 5};    // Vector mit 5 Elementen  
Vector v2{1.23, 3.45, 6.7, 8}; // Vector mit 4 Elementen
```

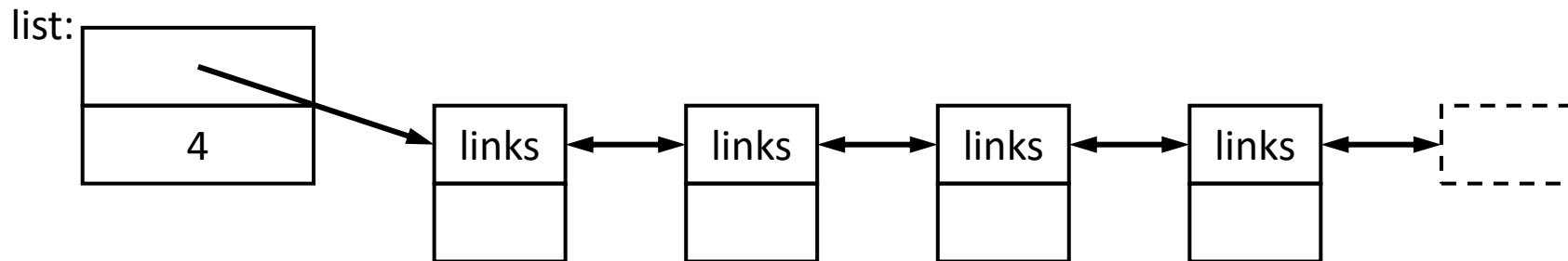
int main()

`.begin()` und `.end()`
spezifizieren den Bereich,
der kopiert werden soll.
Dazu mehr im nächsten
Kapitel „Iteratoren“.

Sequenzielle Container

Standard List

- **std::list** stellt doppelt verknüpfte Liste dar
 - Bietet sich für Sequenzen an, bei denen Elemente hinzugefügt und entfernt werden sollen, ohne dass die anderen Elemente bewegt werden müssen. (z.B. Telefonbuch)
 - Auf Elemente kann nicht durch Indizierung zugegriffen werden. Stattdessen muss die Liste durchlaufen werden.
 - Performt schlecht beim Traversieren, Sortieren und Suchen



```
#include <list>
```

```
std::list<Entry> phonebook = {
    {"David Hume",123456},
    {"Karl Popper",234567},
    {"Bertrand Russell",345678}
};
```

```
uint32_t getNumber(const std::string& s) {
    for (const auto& x : phonebook)
        if (x.name == s)
            return x.number;
    return 0; // 0 steht für "Nummer nicht gefunden"
}
```

Sequenzielle Container

Zusammenfassung



- Sequenzielle Container verwalten Elemente eines beliebigen Datentyps.
 - Die Reihenfolge der Elemente im Container werden durch den Programmierer festgelegt
 - Es gibt grundlegende Funktionalitäten, die für alle sequenzielle Container unabhängig vom Datentyp der Elemente implementiert sind
- **std::vector** ist ein sequenzieller Container, dessen Elemente alle hintereinander im Speicher allokiert sind.
 - Ist der Container für die meisten Anwendungsfälle
 - Allokiert mehr Speicher als Elemente vorhanden sind, um den überschüssigen Speicher für neue Elemente verwenden zu können
- **std::initializer_list** bietet sich an, um eigene Klassen direkt mit einer Liste an Elementen initialisieren zu können.

Sequenzielle Container

Verständnisfragen

PIN: 296714

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Die feste Größe des **`std::array`** muss dem Compiler zur Kompilierungszeit bekannt sein.
- Ein **`std::vector`** allokiert immer genau so viel Speicher wie für die Elemente benötigt wird.
- Die Speicheradressen der Elemente eines **`std::vector`** können sich zur Laufzeit mehrmals ändern.
- Die Reihenfolge der Elemente einer **`std::list`** stimmt immer mit ihrer Reihenfolge im Speicher überein.
- Die Reihenfolge der Elemente eines **`std::vector`** stimmt immer mit ihrer Reihenfolge im Speicher überein.
- Bei sequenziellen Containern der Standard-Bibliothek wird immer ein Fehler geworfen, wenn versucht wird, mit dem Indizierungsoperator `[]` auf ein nicht existierendes Element zuzugreifen.



Iteratoren

Lernziele

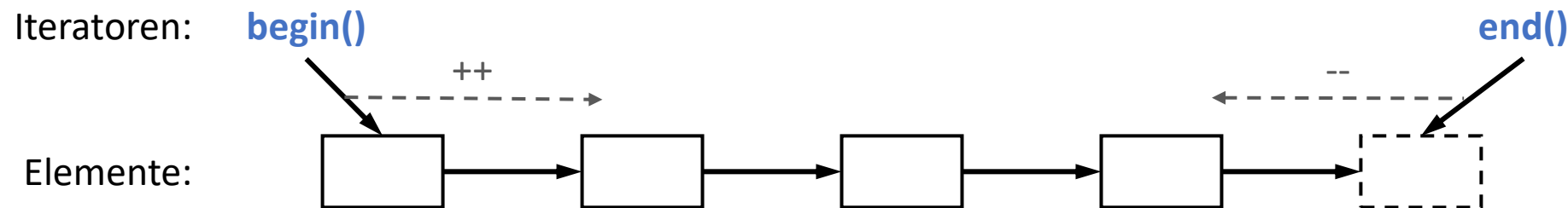
- Es ist bekannt, worum es sich bei einem Iterator handelt.
- Der **Nutzen** und die **Funktionsweise** von Iteratoren wurde verstanden.
 - Iteratoren können verwendet werden, um Elemente in Standard Containern hinzuzufügen und aus ihnen zu entfernen.
 - Algorithmen auf Standard Containern, die Iteratoren verwenden, können korrekt benutzt werden.
- Für eigene Container-Klassen können Iteratoren erstellt werden.

Iteratoren



Nutzen

- Bisher können wir noch keine Elemente im `std::vector` und in der `std::list` an beliebiger Stelle hinzufügen oder entfernen
- Des Weiteren werden Container selten nur zum Aufbewahren von Daten verwendet
 - Häufig müssen die Elemente sortiert, ausgegeben, gesucht, gefiltert, usw. werden
 - Teilweise möchte man nur eine Teilmenge der Daten erhalten oder kopieren
- Um Elemente einzufügen und zu löschen und Algorithmen auf den Daten auszuführen, besitzen viele STL-Container sogenannte **Iteratoren**
 - Iteratoren können als Pointer auf Elemente eines Containers mit erweiterten Funktionalitäten aufgefasst werden.
 - Sie können inkrementiert werden, um auf benachbarte Elemente zu zeigen.



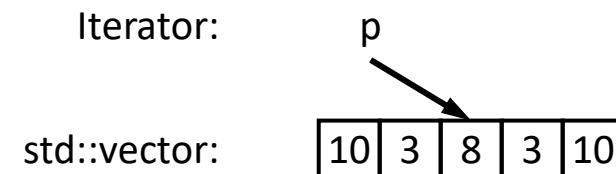
Iteratoren

Funktionsweise

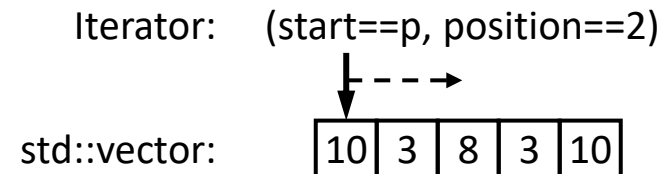


- Um Elemente einzufügen und zu löschen und Algorithmen auf den Daten auszuführen, besitzen viele STL-Container sogenannte **Iteratoren**
 - Iteratoren können je nach Container unterschiedlich implementiert sein, bieten jedoch immer dieselben Funktionalitäten
 - Iteratoren trennen Algorithmen auf den Containern von der Datenverwaltung
 - Die Algorithmen müssen dadurch nicht von jedem Container implementiert werden und nicht wissen, welche Daten der Container verwaltet

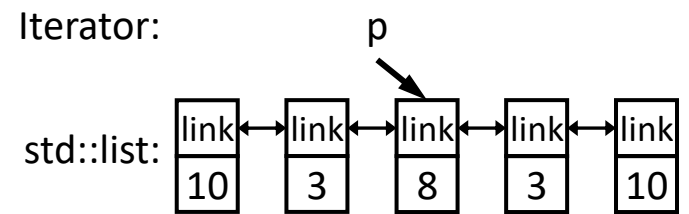
Mögliche Umsetzung eines `std::vector::iterator` als Pointer auf ein beliebiges Element:



Mögliche Umsetzung eines `std::vector::iterator` als Pointer auf das erste Element plus Versatz:



Mögliche Umsetzung eines `std::list::iterator` als Pointer auf ein Link-Element:





Iteratoren

Mindestanforderungen

■ Für einen Iterator gelten **Mindestanforderungen:**

- Ein Iterator **referenziert** ein Element
- Über den *Operator kann auf das referenzierte Element **zugegriffen** werden
- Ein Iterator kann **inkrementiert** werden
- Iteratoren können auf **Gleichheit** verglichen werden
- Iteratoren können auf **Unterschiedlichkeit** verglichen werden

```
class Vector {  
public:  
    /* ...Vector Konstruktor & Methoden... */  
    // Iterator für Vector  
    class iterator {  
        double* ptr; // Referenziertes Element  
    public:  
        explicit iterator(double& elem) : ptr(&elem) {}  
        double& operator*() const { return *ptr; }  
        iterator& operator++() { ptr += 1; return *this; }  
        iterator operator++(int)  
        { iterator retval = *this; ++(*this); return retval; }  
        bool operator==(iterator other) const { return ptr == other.ptr; }  
        bool operator!=(iterator other) const { return !(*this == other); }  
    };  
private:  
    /* ...Vector Member-Variablen... */  
};
```

Iteratoren



Iteration mit Iteratoren

- Durch die Mindestanforderungen an Iteratoren spannt ein Iteratoren-Paar immer einen Bereich über die Elemente eines Containers auf.
 - Kann verwendet werden, um über die Elemente des Containers zu **iterieren**
 - Iteration erfolgt vom Element des ersten Iterators bis zum Element vor dem zweiten Iterator

```
int main()
{
    Vector v1{ 2.5, 1.4, 3.14, -1.8 };

    Vector::iterator beginIt{ v1[0] };
    Vector::iterator endIt{ v1[3] };
    for (; beginIt != endIt; ++beginIt)
        std::cout << *beginIt << '\n';
}
```

Ausgabe:

```
2.5
1.4
3.14
```

Iteratoren



Iteration mit Iteratoren

- Bietet eine Container-Klasse die Methoden ***begin()*** und ***end()***, um einen Iterator auf das erste und einen hinter das letzte Element des Containers zu erhalten, kann der **Range-For Loop** verwendet werden.
 - Soll der Range-For Loop mit einer konstanten Vector-Instanz verwendet werden können, müssen die Methoden *begin()* und *end()* auch als konstante Member-Funktionen implementiert sein.
 - Viele Container-Klassen implementieren zusätzlich einen ***const_iterator***, bei dem der *Operator eine const-Reference zurückgibt.

```
class Vector {  
public:  
    /* ... */  
    class iterator { /* ... */ };  
    iterator begin() { return iterator{ m_elements[0] }; }  
    iterator end() { return iterator{ m_elements[m_size] }; }  
private:  
    double* m_elements; const unsigned m_size;  
};
```

```
int main()  
{  
    Vector v1{ 2.5, 1.4, 3.14, -1.8 };  
  
    for (auto& elem : v1)  
        std::cout << elem << '\n';  
}
```

Ausgabe:

```
2.5  
1.4  
3.14  
-1.8
```

Iteratoren

Einfügen von Elementen

- Mit ***insert()*** wird ein Element vor den angegebenen Iterator kopiert.
- Alternativ wird mit ***insert()*** wird durch Iteratoren ein Bereich von Elementen kopiert.
- **C++11** Mit ***emplace()*** wird ein neues Element direkt im Container vor dem angegebenen Iterator erstellt.
- ***insert()*** und ***emplace()*** geben den neuen Iterator auf das erste hinzugefügte Element zurück.



```
std::vector<Entry> phonebook = {"David Hume", 123456},  
                                {"Karl Popper", 234567}, {"Bertrand Russell", 345678};
```

```
// Iterator auf das zweite Element  
auto it = ++phonebook.begin();
```

```
// Füge Bernd vor dem zweiten Element ein  
it = phonebook.insert(it, Entry{ "Bernd LR", 49152391 });
```

```
std::vector<Entry> newEntries = {  
    {"Perry Schnabel", 789654}, {"Heinz Doof", 321456 }  
};
```

```
// Füge neue vector-Elemente vor dem zweiten Element ein  
it = phonebook.insert(it, newEntries.begin(), newEntries.end());
```

```
// Füge ein neues Element vor dem dritten Element ein  
phonebook.emplace(++it, "Gustav Gans", 24680);  
for (auto p = phonebook.begin(); p != phonebook.end(); ++p)  
    std::cout << *p << '\n';
```

```
David Hume: 123456  
Perry Schnabel: 789654  
Gustav Gans: 24680  
Heinz Doof: 321456  
Bernd LR: 49152391  
Karl Popper: 234567  
Bertrand Russell: 345678
```



Iteratoren

Finden und Entfernen

- **Finden und Entfernen** von Elementen
 - ***erase()*** entfernt das Element, auf das der Iterator zeigt, und gibt den Iterator auf das Element dahinter zurück.
- **Reverse Iteratoren**
 - ***rbegin()*** und ***rend()*** traversieren den Container von hinten nach vorne

```
std::vector<Entry> phonebook = {  
    {"David Hume",123456},  
    {"Karl Popper",234567},  
    {"Bertrand Russell",345678}  
};  
// Lösche das letzte Element  
phonebook.erase(--phonebook.end());  
  
// Durchlaufe den Vektor von hinten nach vorne  
for (auto p = phonebook.rbegin(); p != phonebook.rend(); ++p)  
    std::cout << *p << '\n';
```

```
Karl Popper: 234567  
David Hume: 123456
```

Iteratoren

Finden und Entfernen

■ Finden und Entfernen von Elementen

- ***erase()*** entfernt das Element, auf das der Iterator zeigt, und gibt den Iterator auf das Element dahinter zurück.
- Um Elemente in Schleifen zu entfernen, muss auf die **Invalidierung** der Iteratoren geachtet werden!



```
std::vector<int> values = { 10, 3, 8, 3, 10, 9 };

auto endIt = values.end();
for (auto it = values.begin(); it != endIt; )
{
    std::cout << "Schleifenelement: " << *it << '\n';
    if (*it == 10)
    {
        // Die bisherigen Iteratoren sind
        // nach dem erase nicht mehr gültig
        it = values.erase(it);
        endIt = values.end();
    }
    else
        ++it;
}
std::cout << "Vector: ";
for (const auto& i : values)
    std::cout << i << ' ';
```

```
Schleifenelement: 10
Schleifenelement: 3
Schleifenelement: 8
Schleifenelement: 3
Schleifenelement: 10
Schleifenelement: 9
Vector: 3 8 3 9
```



Iteratoren

Finden und Entfernen

- **Finden und Entfernen** von Elementen
 - ***erase()*** entfernt das Element, auf das der Iterator zeigt, und gibt den Iterator auf das Element dahinter zurück.
 - Um Elemente in Schleifen zu entfernen, muss auf die **Invalidierung** der Iteratoren geachtet werden!
 - Mit ***std::find()*** wird nach dem ersten Element eines Bereiches mit einem bestimmten Wert gesucht.
 - Nutzt den **==**-operator zum Vergleich mit den Elementen

```
std::vector<int> values = { 10, 3, 8, 3, 10, 9 };
auto it = values.begin();
auto endIt = values.end();
while (it != endIt) {
    // Suche das erste Element mit Wert 10
    // im Bereich [it,endIt)
    it = std::find(it, endIt, 10);
    if (it != endIt) {
        std::cout << "Schleifenelement: " << *it << '\n';
        // Die bisherigen Iteratoren sind
        // nach dem erase nicht mehr gültig
        it = values.erase(it);
        endIt = values.end();
    }
    else
        std::cout << "Schleifenelement: end()" << '\n';
}
std::cout << "Vector: ";
for (const auto& i : values)
    std::cout << i << ' ';
```

```
Schleifenelement: 10
Schleifenelement: 10
Schleifenelement: end()
Vector: 3 8 3 9
```

Iteratoren

Finden und Entfernen



■ Finden und Entfernen von Elementen

- ***erase()*** entfernt das Element, auf das der Iterator zeigt, und gibt den Iterator auf das Element dahinter zurück.
- Um Elemente in Schleifen zu entfernen, muss auf die **Invalidierung** der Iteratoren geachtet werden!
- Mit ***std::find()*** wird nach dem ersten Element eines Bereiches mit einem bestimmten Wert gesucht.
- **C++20** Mit ***std::erase()*** werden Elemente aus einem Bereich mit einem bestimmten Wert entfernt.
 - Nutzt den **==**operator zum Vergleich mit den Elementen

```
std::vector<int> values = { 10, 3, 8, 3, 10, 9 };

// Entferne die Elemente mit Wert 10
auto anzahl = std::erase(values, 10);
std::cout << anzahl << " Elemente wurden entfernt.\n";

std::cout << "Vector: ";
for (const auto& i : values)
    std::cout << i << ' ';
```

```
2 Elemente wurden entfernt.
Vector: 3 8 3 9
```

Iteratoren



Sortieren und Kopieren von Elementen

- Mit ***std::sort()*** werden die Werte in einem Bereich sortiert, sofern ein entsprechender Vergleichsoperator für den Datentyp existiert.
- Um Elemente in einen *std::vector* oder einer *std::list* via Iteratoren zu kopieren, wird der ***std::back_inserter()*** benötigt
 - Mit ***std::unique_copy()*** wird jeder Wert nur einmal kopiert => Es gibt keine Duplikate

```
std::list<int> valuesList(std::vector<int>& vec)
{
    std::list<int> list;
    // Sortiere den Vektor aufsteigend
    std::sort(vec.begin(), vec.end());
    // Kopiere die Werte einzigartig in die Liste
    std::unique_copy(vec.begin(), vec.end(),
                    std::back_inserter(list));
    return list;
}
```

```
int main() {
    std::vector<int> values = { 10, 3, 8, 3, 10, 9 };
    auto list = valuesList(values);
    std::cout << "Vector: ";
    for (const auto& i : values)
        std::cout << i << ' ';
    std::cout << "\nList: ";
    for (const auto& i : list)
        std::cout << i << ' ';
}
```

```
Vector: 3 3 8 9 10 10
List: 3 8 9 10
```

Iteratoren

Zusammenfassung



- Iteratoren **verweisen auf ein Element** in einer Reihe von Elementen.
- Iteratoren werden **inkrementiert**, um auf das nächste Element zu zeigen.
- Der **Range-For Loop** verwendet intern Iteratoren, um ohne potenzielle Bereichsfehler über Elemente eines Containers zu iterieren.
- Über Iteratoren werden Elemente an bestimmten Stellen in Standard Containern **hinzugefügt** und **entfernt**.
- Mithilfe von Iteratoren können viele **Algorithmen** verwendet werden, ohne dass für den Algorithmus bekannt sein muss, von welchem Container der Iterator stammt und auf welchen Typ von Element er verweist.
 - Zum Beispiel, um Elemente eines beliebigen Containers zu finden
 - oder um einen beliebigen Container zu sortieren

Iteratoren

Verständnisfragen

PIN: 296714

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Für Standard Container gibt die Funktion **.end()** einen Iterator auf das letzte Element im Container zurück.
- Der Range-For Loop verwendet Iteratoren, um über die Elemente eines Containers zu iterieren.
- **insert()** kann ein neues Objekt direkt in einem Container erzeugen, ohne, dass das Objekt kopiert werden muss.

■ Was ist ein Iterator in C++?

- Ein Konstruktor für Container
- Ein Zeiger auf das erste Element in einem Container
- Ein Objekt, das den Zugriff auf die Elemente in einem Container ermöglicht
- Ein spezieller Standard Container

■ Welche dieser Funktionen für einen `std::vector` benötigen zwingend Iteratoren?

- `emplace_back()`
- `erase()`
- `insert()`
- `at()`



Assoziative Container

Lernziele

- Am Ende dieses Kapitels wurden Assoziative Container grundlegend verstanden
 - Wie Elemente in assoziativen Containern sortiert werden, wurde verstanden und verschiedene Möglichkeiten zum Sortieren sind bekannt
 - Es können eigene Vergleichsfunktionen für assoziative Container erstellt werden

Assoziative Container

Ordnungsrelationen



```
#include <set>
```

- Die Standardbibliothek bietet Möglichkeiten, Container zu sortieren
 - Die Funktion `std::sort(begin, end)` sortiert u.a. die Elemente eines `std::vector`, sofern diese in eine Reihenfolge gebracht werden können
- Um für eigene Klassen geeignete Sortierungen zu erzeugen, möchten wir uns verschiedene Arten von Sortierungen, sogenannte **Ordnungsrelationen** anschauen.
- Sortiert die folgenden Elemente dazu aufsteigend:

c: **72.117.110.100**

e: **104.117.104.110**

a: **72.097.117.115**

d: **72.097.117.115**

b: **102.105.115.099.104**

f: **75.097.116.122.101**

Assoziative Container

Ordnungsrelationen



Strenge Totalordnung

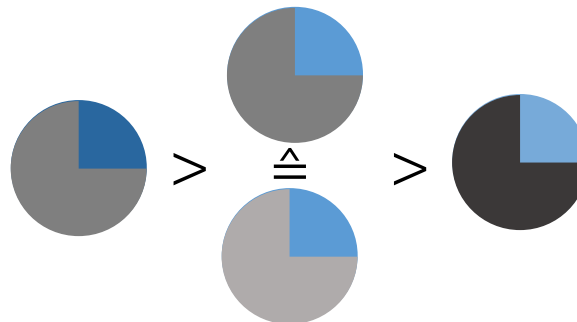
- Definiert eine eindeutige Reihenfolge
- Für jedes Elementpaar gilt:
 - entweder $x < y$ oder $x = y$ oder $x > y$
 - Impliziert **Substitution**: Sind a und b äquivalent, so auch $f(a)$ und $f(b)$ für jede mögliche Funktion f



Beispiel: Sortierung nach Blauton.

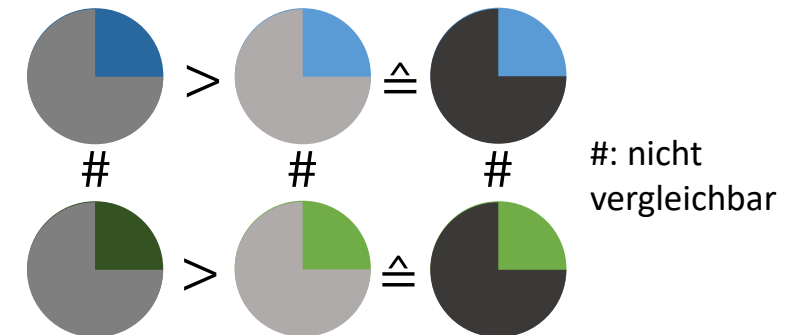
Strenge schwache Ordnung

- Erlaubt mehrere gleichartige Objekte
- Für jedes Elementpaar gilt:
 - entweder $x < y$ oder $x \triangleq y$ oder $x > y$
 - Impliziert **nicht** Substitution



Halbordnung

- Elemente können miteinander unvergleichbar sein
- Für jedes Elementpaar gilt:
 - $x \not< y \wedge x \not> y \not\Rightarrow x \triangleq y$
 - Impliziert **nicht** Substitution



Assoziative Container



Ordnungsrelationen – Spaceship Operator

- **C++20** Durch das Überschreiben des **Spaceship-Operators** `<=>` wird für eine eigene Klasse die Sortierung spezifiziert.

(https://en.cppreference.com/w/cpp/language/default_comparisons)

- Dazu kann als Rückgabetyp des Spaceship-Operators eine der drei Ordnungsrelationen angegeben werden: **`std::strong_ordering`**, **`std::weak_ordering`**, **`std::partial_ordering`**

```
class Person {  
public:  
    Person(uint8_t id, std::string name);  
    uint8_t getID() const;  
    std::string getName() const;  
private:  
    uint8_t m_id;  
    std::string m_name;  
};
```

```
std::weak_ordering operator<=>(const Person& p1, const Person& p2)  
{  
    if (p1.getID() < p2.getID())  
        return std::weak_ordering::less;  
  
    else if (p1.getID() > p2.getID())  
        return std::weak_ordering::greater;  
  
    return std::weak_ordering::equivalent;  
}
```

Assoziative Container



Ordnungsrelationen

- Wir können die Elemente einfach in eine Reihenfolge bringen, indem wir sie als Zahlen interpretieren und der Größe nach anordnen.
- Dadurch haben wir automatisch/intuitiv eine **Strenge Totalordnung** erstellt:
 - Alle Elemente sind miteinander vergleichbar
 - Wir können eine klare Reihenfolge bestimmen
 - Egal, welche deterministische Funktion wir auf die Elemente mit demselben Zahlenwert anwenden, sie werden danach immer noch gleich sein.

kleinstes Element

a,d:	72.097.117.115
c:	72.117.110.100
e:	104.117.104.110
f:	75.097.116.122.101
b:	102.105.115.099.104

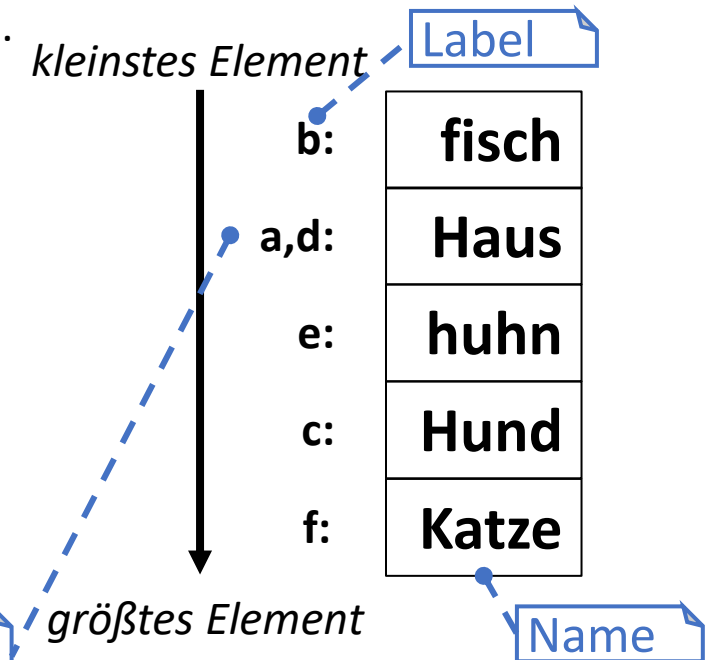
größtes Element

Assoziative Container



Ordnungsrelationen

- Aber was ist, wenn die Elemente gar keine Zahlen sind, sondern eine Reihe von ASCII-Zeichen?
 - Wir könnten die **Namen** nun alphabetisch sortieren und dabei nicht zwischen Groß- und Kleinschreibung unterscheiden.
 - Dann würden **Huhn** und **huhn** als gleichwertig angesehen werden, obwohl sie sich unterscheiden.
`name[0]` würde jedoch ein unterschiedliches Zeichen zurückgeben.
 - Es handelt sich um eine **Strenge schwache Ordnung**.
 - Stellen wir uns jetzt vor, es gäbe eine Funktion, mit der wir auf das **Label** (den Buchstaben) des Elements zugreifen können:
 - Dann können wir diese Buchstaben mit derselben alphabetischen Sortierung wie zuvor sortieren: $a < b < c < d < e < f$
 - Lassen wir eine solche Funktion zu, so wären die Elemente **a** und **d**, die zuvor äquivalent waren, nun nicht mehr äquivalent.
 - Auch dann handelt es sich um eine **Strenge schwache Ordnung**.



Assoziative Container

Ordnungsrelationen



- Aber was ist, wenn die Elemente gar keine ASCII-Zeichen sind, sondern (mathematische) Vektoren aus dem $\mathbb{R}^4$ und dem $\mathbb{R}^5$?
 - Wir könnten definieren, dass unterschiedlich dimensionale Vektoren nicht miteinander vergleichbar sein sollen.
 - Dadurch hätten wir Elemente, die miteinander unvergleichbar (also weder kleiner, noch größer, noch gleich) sind.
 - Indem wir die Länge der Vektoren bestimmen, können wir die gleichdimensionalen Vektoren untereinander in eine Reihenfolge bringen.
 - Dadurch erhalten wir eine **Halbordnung**.

kleinstes Element

	$\mathbb{R}^4$		$\mathbb{R}^5$		
a,d:	$\left\ \begin{pmatrix} 72 \\ 97 \\ 117 \\ 115 \end{pmatrix} \right\ $	≈ 203.73	f:	$\left\ \begin{pmatrix} 75 \\ 97 \\ 116 \\ 122 \\ 101 \end{pmatrix} \right\ $	≈ 231.46
c:	$\left\ \begin{pmatrix} 72 \\ 117 \\ 110 \\ 115 \end{pmatrix} \right\ $	≈ 210.23			
e:	$\left\ \begin{pmatrix} 104 \\ 117 \\ 104 \\ 110 \end{pmatrix} \right\ $	≈ 217.76	b:	$\left\ \begin{pmatrix} 102 \\ 105 \\ 115 \\ 99 \\ 104 \end{pmatrix} \right\ $	≈ 235.10

größtes Element

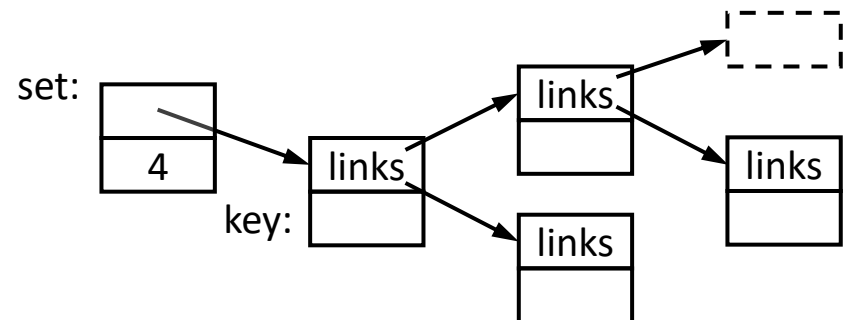
Standard Set

Funktionsweise



```
#include <set>
```

- Damit Elemente von Beginn an sortiert sind und neue Elemente richtig einsortiert werden, bietet die STL den Container **std::set** an
 - `std::set` ist eine geordnete Liste mit einzigartigen Elementen, genannt Keys
 - Die Sortierung findet über eine „key comparison“ Funktion „Compare“ statt
 - Compare ruft den Operator() für die Elemente auf, um zwei Elemente miteinander zu vergleichen.
 - Wenn das erste Element vor dem zweiten gelistet werden soll, wird `true` zurückgegeben.
 - Zwei Elemente gelten als identisch, wenn Compare für `(elem1, elem2)` und `(elem2, elem1)` jeweils `false` zurück gibt.
 - `std::set` ist als balancierter Binärbaum implementiert



Standard Set

Hinzufügen und Entfernen von Elementen



```
#include <set>
```

■ Hinzufügen von Elementen

- Mit ***insert()*** können sowohl einzelne Elemente als auch mehrere Elemente hinzugefügt werden.
- **C++11** Mit ***emplace()*** wird ein neues Element ohne unnötige Copy- & Move-Operationen hinzugefügt.

■ Entfernen von Elementen

- Mit ***erase()*** werden Elemente mit einem bestimmten Wert entfernt.
- Oder via Angabe eines Iterators oder eines Bereichs durch ***begin*** und ***end***
 - Es wird ein Iterator auf das Element nach dem zuletzt gelöschten zurückgegeben.

```
std::set<int> values = { 1, -5, 3, 2, -4, 10, 6, 2 };  
for (auto i : values)  
    std::cout << i << " ";  
std::cout << '\n';
```

```
-5 -4 1 2 3 6 10
```

```
std::set<int> newVals = { 11, 9, 200, 0 };  
// Füge alle Elemente bis auf das letzte hinzu  
values.insert(newVals.begin(), --newVals.end());  
// Entferne das Element mit dem Wert -4  
values.erase(-4);  
// Entferne das Element an erster Position  
// und gib das neue erste Element aus  
std::cout << "Kleinster Wert: " <<  
*values.erase(values.begin()) << '\n';  
// Füge das Element mit Wert -5 hinzu  
values.emplace(-5);  
for (auto i : values)  
    std::cout << i << " ";
```

```
Kleinster Wert: 0
```

```
-5 0 1 2 3 6 9 10 11
```

Standard Set

Mutable – Motivation



```
#include <set>
```

- Objekte in Sets **können nicht verändert werden**, da dies einen Einfluss auf die Reihenfolge haben könnte.
 - Selbst, wenn eine Member-Variable geändert werden soll, die für den Vergleich nicht relevant ist. (→ Strenge Schwache Ordnung)

```
struct Money {  
    int euro = 0;  
    int cent = 0;  
    int year = 0;  
    bool operator<(const Money& other) const {  
        return euro == other.euro  
            ? cent < other.cent  
            : euro < other.euro;  
    }  
};
```

```
std::set<Money> moneySet = {  
    {10,25,2019},{10,25,2018},{20,10,2019},{10,50,2017}  
};
```

```
std::cout << "Geld Menge:\n";  
for (const auto& g : moneySet)  
    std::cout  
        << "{" << g.euro << "." << g.cent  
        << " (" << g.year << ")}\n";
```

```
Geld Menge:  
{10.25 (2019)}  
{10.50 (2017)}  
{20.10 (2019)}
```

```
// ERROR: Der Ausdruck muss ein änderbarer lvalue sein  
moneySet.begin()->year = 2020;
```

Standard Set

Mutable



```
#include <set>
```

- Objekte in Sets können nicht verändert werden, da dies einen Einfluss auf die Reihenfolge haben könnte.
- Mit dem Schlüsselwort ***mutable*** werden die Attribute deklariert, die jederzeit geändert werden dürfen.
 - Wird ein Attribut mit mutable verändert, sollte sich die Sortierung des Sets nicht ändern.
 - Es muss sich dann um eine Strenge Schwache Ordnung handeln.

```
struct Money {  
    int euro = 0;  
    int cent = 0;  
    mutable int year = 0;  
    bool operator<(const Money& other) const {  
        return euro == other.euro  
            ? cent < other.cent  
            : euro < other.euro;  
    }  
};
```

```
std::set<Money> moneySet = {  
    {10,25,2019}, {10,25,2018}, {20,10,2019}, {10,50,2017}  
};  
moneySet.begin()->year = 2020;  
  
std::cout << "Geld Menge:\n";  
for (const auto& g : moneySet)  
    std::cout  
        << "{" << g.euro << "." << g.cent  
        << " (" << g.year << ")}\n";
```

```
Geld Menge:  
{10.25 (2020)}  
{10.50 (2017)}  
{20.10 (2019)}
```

Vergleichsoperatoren

Standard Compare Operator



```
#include <set>
```

- `std::set` verwendet zum Vergleichen der Elemente standardmäßig das Funktionsobjekt **`std::less`**, welches für die Objekte den **`operator<`** aufruft.
 - `std::less::operator()` erhält zwei Elemente vom Datentyp der Set-Elemente (lhs sei das erste Element, rhs sei das zweite)
 - Wenn der Datentyp kein Pointer ist, wird `true` zurückgegeben, wenn `lhs < rhs`, und sonst `false`
 - Mit **`std::greater`** werden die Elemente absteigend sortiert.

Sortiert die Integer-Werte aufsteigend.

```
std::set<int, std::less<int>> values  
= { 1, -5, 3, 2, -4, 10, 6, 2 };  
for (auto i : values)  
    std::cout << i << " ";
```

-5 -4 1 2 3 6 10

Sortiert die Float-Werte absteigend. Jedoch nur den ersten Wert bei selbem Ganzzahlwert.

```
std::set<float, std::greater<int>> rounded =  
{ 1.4f, 2.3f, 2.1f, 0.5f, 1.9f, -0.2f };  
for (auto f : rounded)  
    std::cout << f << " ";
```

2.3 1.4 0.5

Vergleichsoperatoren



Pointer

- Ist der Datentyp ein **Pointer**, wird *true* zurückgegeben, wenn lhs in der implementierungs-spezifischen **strengen Totalordnung** vor rhs kommt, und sonst *false*.
 - Normalerweise: Vergleich der **Speicheradressen** und nicht der Elemente, auf die gezeigt wird

```
auto three = new int(3);
auto two = new int(2);
auto one = new int(1);
auto three2 = new int(3);
std::set<int*, std::greater<int*>> ptr
    = { one, three, two, three2, two };

for (const auto& p : ptr)
    std::cout << p << "{" << *p << "} ";
delete one, two, three, three2;
```

```
000002828B102FC0{3} 000002828B0FB1A0{3} ...
... 000002828B0FB160{1} 000002828B0FB120{2}
```

Vergleichsoperatoren

Compare Operator für Smart Pointer



```
#include <set>
```

- Für Shared Pointer sind die Vergleichsoperatoren überschrieben
 - Für zwei Shared Pointer p1 und p2 vergleicht $p1 < p2$ die **Adresswerte** der Pointer
 - $p1 < p2$ ist äquivalent mit $p1.get() < p2.get()$
 - Für Weak Pointer sind die Vergleichsoperatoren **nicht** definiert => Compilerfehler

```
using int_ptr_t = std::shared_ptr<int>; // Typ-Alias
// Funktion zum verkürzten Schreiben von std::make_shared<int>
int_ptr_t share(int i) { return std::make_shared<int>(i); };
```

```
int main() {
    // Sortiert die Pointer nach den Adresswerten
    std::set<int_ptr_t, std::less<int_ptr_t>> ptr =
        { share(3), share(10), share(1), share(-1), share(3) };
    for (const auto& p : ptr)
        std::cout << p << "{" << *p << "}";
}
```

```
000002BEBB12FDD0{10} 000002BEBB12FE30{3} ...
... 000002BEBB1302B0{-1} 000002BEBB130310{3} ...
... 000002BEBB1304F0{1}
```

Vergleichsoperatoren

Custom Compare Operator



```
#include <set>
```

- Damit der Compare Operator geordneter Container die Objekte, auf die die Pointer zeigen, berücksichtigen, benötigt es eine **eigene Funktion**.
 - Meistens genügt es dazu, eine **Operator-Überladung** zum **Vergleich der Pointer** zur Verfügung zu stellen

```
struct Money {  
    int euro = 0; int cent = 0;  
    std::strong_ordering operator<=>  
        (const Money& other) const  
    {  
        return euro == other.euro  
            ? cent <=> other.cent  
            : euro <=> other.euro;  
    }  
};
```

```
auto operator<=>  
(const std::shared_ptr<Money>& lhs, const std::shared_ptr<Money>& rhs)  
{  
    return *lhs <=> *rhs;  
}  
  
int main() {  
    std::set<std::shared_ptr<Money>> moneySet{  
        std::make_shared<Money>(20, 10),  
        std::make_shared<Money>(20, 10),  
        std::make_shared<Money>(10, 50)  
    };  
    std::cout << "Shared Pointer Set auf Money: ";  
    for (const auto& p : moneySet)  
        std::cout << "{" << p->euro << "." << p->cent << " }";  
}
```

std::set findet die eigene Überladung für std::shared_ptr<Money>

```
Shared Pointer Set auf Geld: {10.50} {20.10}
```

Vergleichsoperatoren

Custom Compare Operator



```
#include <set>
```

- Damit der Compare Operator geordneter Container die Objekte, auf die die Pointer zeigen, berücksichtigen, benötigt es eine **eigene Funktion**.
 - **C++11** Mithilfe eines **Lambda-Ausdrucks** kann eine Vergleichsfunktion implizit an der Stelle definiert werden, an der sie benötigt wird (mehr dazu in einer späteren Vorlesung)

In der Main-Funktion wird eine Vergleichsfunktion erstellt und übergeben.

```
int main() {  
    auto cmp =  
    [ ](const std::shared_ptr<Money>&lhs, const std::shared_ptr<Money>&rhs)  
    { return *lhs < *rhs; };  
  
    std::set<std::shared_ptr<Money>, decltype(cmp)> moneySet(cmp);  
  
    moneySet.insert({  
        std::make_shared<Money>(20, 10),  
        std::make_shared<Money>(20, 10),  
        std::make_shared<Money>(10, 50)  
    });  
    /* Ausgabe */  
}
```

C++20 Es reicht
`moneySet(cmp);` zu schreiben!

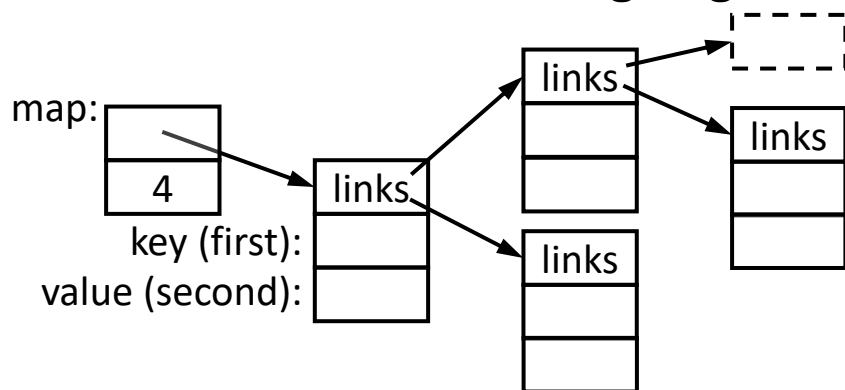
Weitere Möglichkeiten und Erklärungen zur Angabe von Custom Compare Operatoren erläutert der Nutzer *diralik* auf Stack Overflow:
<https://stackoverflow.com/a/46128321>

Shared Pointer Set auf Geld: {10.50} {20.10}

Standard Map

Funktionsweise

- Die `std::map` erweitert das `std::set` neben dem Key um ein zweites Element, welches dem Key einen Wert zuordnet
 - In anderen Programmiersprachen wird der Container als „dictionary“ oder „associative array“ bezeichnet
 - `std::map` ist für das Suchen und Einfügen von Werten optimiert (Suchzeit in $O(\log(n))$)
- Wird ein Index aufgerufen, der noch nicht existiert, wird ein neues Element mit diesem Key und dem Default-Wert angelegt.



Containertyp

Datentyp des Schlüssels

Datentyp der Werte

Optional: Vergleichsfunktion

`std::map<std::string, uint32_t, std::less<std::string>>`



```
#include <map>
```

```
std::map<std::string, uint32_t>
```

```
phonebook = { {"David Hume", 123456},  
              {"Karl Popper", 234567},  
              {"Bertrand Russell", 345678}
```

```
};
```

```
// Erstellt einen neuen Eintrag, wenn s nicht existiert
```

```
uint32_t getOrCreateNumber(const std::string& s) {
```

```
    return phonebook[s];
```

```
}
```

```
// Geht sicher, dass s existiert
```

```
uint32_t getNumber(const std::string& s) {
```

```
    auto it = phonebook.find(s);
```

```
    if (it != phonebook.end())
```

```
        return it->second;
```

```
    return 0;
```

```
}
```

Standard Map

Werte ohne Standard-Konstruktor



```
#include <map>
```

- Wird ein Index aufgerufen, der noch nicht existiert, wird ein neues Element mit diesem Key und dem **Default-Wert** angelegt.
 - Dazu **muss** ein **Standard-Konstruktor** für das Wertobjekt vorhanden sein!
 - **Ohne Standard-Konstruktor** müssen Elemente mit `.insert()` oder `.emplace()` hinzugefügt werden
 - Der Zugriff auf Elemente ohne Standard-Konstruktor muss via `.find()` (gewährleistet korrekten Programmablauf) oder `.at()` (führt zu undefiniertem Verhalten, wenn das Element nicht existiert) stattfinden.

```
int main() {  
    std::map<uint16_t, Vector> vectorMap;  
    vectorMap[0] = Vector{ 3.0, 1.2, 7.5 }; // ERROR: Kein Standardkonstruktor vorhanden  
    vectorMap.emplace(0, std::initializer_list{ 3.0, 1.2, 7.5 }); // Fügt einen Vector hinzu  
    std::cout << "Vector: 0, Element: 2 -> " << vectorMap[0][2] << '\n'; // ERROR: Kein Standardkonstruktor!  
    std::cout << "Vector: 0, Element: 2 -> " << vectorMap.at(0)[2] << '\n';  
}
```

Vector besitzt nur den
Konstruktor mit Initializer-List.

```
Vector: 0, Element: 2 -> 7.5
```

Standard Map

Hinzufügen von Elementen



```
#include <map>
```

- *emplace()* findet häufig keinen passenden Konstruktor
 - Die Initialisierungsliste muss als solche angegeben werden
 - **C++17** Mit *try_emplace()* können Konstruktoren mit mehreren Argumenten aufgerufen werden

Funktioniert **nicht**:

```
std::map<uint16_t, Vector> vectorMap;  
vectorMap.emplace(0, { 3.0, 1.2, 7.5 });  
// ERROR: Keine Instanz von emplace  
// stimmt mit Argumenten überein  
vectorMap.emplace(2, 3, 2.4);  
// ERROR: Keine überladene Funktion  
// akzeptiert 3 Elemente
```

Funktioniert:

```
std::map<uint16_t, Vector> vectorMap;  
vectorMap.emplace(0, std::initializer_list{ 3.0, 1.2, 7.5 });  
vectorMap.try_emplace(2, 3, 2.4);
```

```
class Vector {  
public:  
    explicit Vector(std::initializer_list<double> list);  
    explicit Vector(const std::size_t& size, double defaultValue);  
    double& operator[](unsigned idx);  
    ~Vector();  
private:  
    double* m_elements;  
    std::size_t m_size;  
};
```

Vector.h

Standard Map

Hinzufügen von Elementen

- *emplace()* findet häufig keinen passenden Konstruktor
 - **Achtung typischer Fehler:** Objekt manuell erzeugen
 - **Problem:** Das Objekt wird nicht mehr In-Place erzeugt, sondern erst **konstruiert** und dann in den Container **kopiert**.
 - Die Kopie ist eine **Shallow Copy**: Der im kopierten Objekt allokierte Speicher wird referenziert.
 - Das Originalobjekt wird danach **zerstört**, wodurch der allokierte Speicher freigegeben wird, der jedoch in der Kopie weiterhin referenziert ist.



Beispiele zur korrekten Nutzung von `emplace()` in diesem Kontext unter:

<https://en.cppreference.com/cpp/container/map/emplace>

```
int main() {  
    std::map<uint16_t, Vector> vectorMap;  
    vectorMap.emplace(0, Vector{3.0, 1.2, 7.5 });  
    vectorMap.emplace(2, Vector{3, 2.4});  
  
    auto searchResult = vectorMap.find(0);  
    bool wasFound = searchResult != vectorMap.end();  
    std::cout << "Element found? "  
                << std::boolalpha << wasFound << '\n';  
    if (wasFound)  
        std::cout << "Vector: 0, Element: 2 -> "  
                    << vectorMap.find(0)->second[2] << '\n';  
    std::cout << "Vector: 2, Element: 4 -> "  
                << vectorMap.at(2)[4] << '\n';  
} // → Absturz, da Speicherplatz erneut freigegeben!
```

#include <map>

Kopiert das Objekt durch den Aufruf von *emplace()*!

find() bei *std::map* gibt einen Iterator auf das Key-Value-Pair zurück.

```
>> Allokierter Speicher an Adresse 0000018149ECF880 wird freigegeben.  
>> Allokierter Speicher an Adresse 0000018149ED2020 wird freigegeben.  
Element found? true  
Vector: 0, Element: 2 -> -1.45682e+144  
Vector: 2, Element: 4 -> -1.45682e+144  
>> Allokierter Speicher an Adresse 0000018149ED2020 wird freigegeben.
```

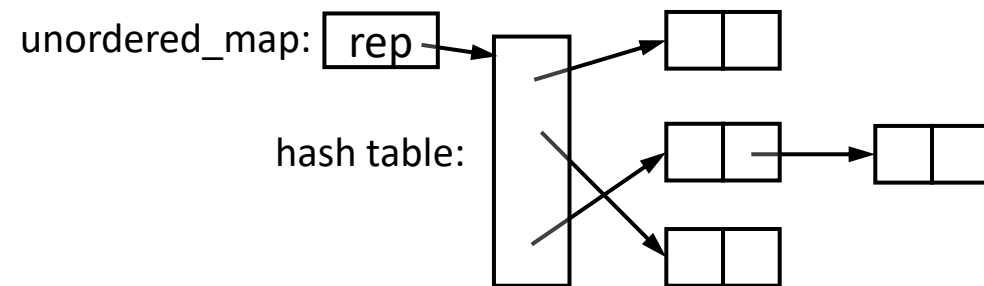
selbe Adresse!

Assoziative Container

Unsortierte Container



- Neben sortierten Containern existieren auch unsortierte (hashed) Container
 - z.B. **`std::unordered_map`** und **`std::unordered_set`**
 - Diese verwenden eine Hash-Funktion, um große Laufzeitvorteile beim Suchen und Einfügen von Elementen zu ermöglichen
 - Hash-Funktionen orientieren sich stark an den zugrundeliegenden Daten, um Optimierungen zu bewirken
 - Mehr dazu in einem möglichen Wahlpflichtmodul Programmieren 3



```
std::unordered_map<std::string, uint32_t> phonebook {  
    {"David Hume", 123456},  
    {"Karl Popper", 234567},  
    {"Bertrand Russell", 345678}  
};
```

Assoziative Container

C++17 Structured Binding



- Mithilfe von **Structured Binding** können Elemente, die in einem Objekt verknüpft sind, jeweils einzeln einem Namen zugeordnet werden
 - Funktioniert für alle Klassenobjekte mit zugreifbaren Member-Variablen
 - Das Schlüsselwort *auto* weist den so deklarierten Variablen den korrekten Datentyp zu
- Structured Binding macht den Code lesbarer und beugt Fehler vor

```
std::map<std::string, uint32_t> phonebook = /*...*/;  
// "Gib mir Name und Nummer aus dem Telefonbuch"  
for (const auto& [name, number] : phonebook)  
    std::cout << name << ": " << number << '\n';
```

```
Bernd LR: 49152391  
Bertrand Russell: 345678  
David Hume: 123456  
Karl Popper: 234567
```

```
std::tuple<std::string, int, double> t1  
    { "Hai", 123, 3.14 };  
auto& [animal, count, price] = t1;  
count = 2;  
std::cout  
    << "Tier: " << animal  
    << ", Anzahl: " << count  
    << ", Preis: " << price;
```

```
Tier: Hai, Anzahl: 2, Preis: 3.14
```

Assoziative Container

Zusammenfassung



- Objekte können in unterschiedliche **Ordnungsrelationen** zueinander gebracht werden.
 - Es ist möglich, dass verschiedene Objekte gleichen Datentyps nicht miteinander vergleichbar sind.
- Assoziative Container **sortieren** den **Schlüssel** eines Elements anhand einer vorgegebenen Sortierung.
 - Standardmäßig werden die Schlüssel **aufsteigend** sortiert.
 - Handelt es sich bei dem Schlüssel um einen **Pointer**, werden meistens die Speicheradressen der Elemente sortiert und nicht die Elemente selbst.
 - Damit eigene Objekte einem assoziativen Container hinzugefügt werden können, müssen Funktionalitäten existieren, mit denen sie sortiert werden können.
 - Attribute der Schlüssel in einem Container können nicht mehr verändert werden, außer einzelne Attribute wurden mit dem Keyword ***mutable*** deklariert.

Assoziative Container

Verständnisfragen

PIN: 296714

<https://poll.hs-kempten.de/poll/>



- **Wofür nutzt man das Schlüsselwort `mutable` in C++?**
 - Gibt es in C++ nicht
 - Für Attribute, die auch in konstanten Methoden geändert werden dürfen
 - Zur Unterdrückung von bestimmten Compiler-Fehlern
 - Für Attribute, die keinen Einfluss auf die Sortierung von Objekten haben
- **Für welche Datentypen des Values einer `std::map` muss ein Vergleichsoperator selbst definiert werden?**
 - Für eigene Klassen
 - Für *int*, *double* und *unsigned*
 - Für *std::weak_pointer*
 - Für *const char**
- **Welche der folgenden Container ermöglichen das automatische Einfügen von Elementen in aufsteigend sortierter Reihenfolge?**
 - *std::vector*
 - *std::map*
 - *std::set*
 - *std::list*

Assoziative Container

Verständnisfragen

PIN: 296714

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Sortierungen in einer Strengen Totalordnung erfüllen immer auch die Anforderungen für eine Halbordnung.
- Der Vergleichsoperator eines `std::set` muss immer mindestens eine Strenge schwache Ordnung erzeugen, um definiertes Verhalten zu gewährleisten.
- Besitzen die Objekte eines `std::set` eine mit **mutable** gekennzeichnete Variable, wird das `std::set` jedes mal neu sortiert, wenn die Variable geändert wird.
- Um ein `std::set` aus Weak Pointern zu erstellen, muss zwingend eine Vergleichsfunktion angegeben werden.
- Bei einer `std::map` kann mit dem Indizierungsoperator `[]` immer auf den zum Schlüssel gespeicherten Wert zugegriffen werden.

Standard-Container



Ratschläge

- Alle Container der STL und ihre Funktionalitäten sind gut und ausführlich dokumentiert
 - Es ist wichtiger, dass man weiß, wonach man suchen muss, als die Syntax auswendig zu können
- Nutze ***std::vector*** als Default Container
- Nutze Range-For-Loops oder die Begin/End-Iteratoren zum Traversieren von Containern
 - Vermeiden performant Bereich-Zugriffs-Fehler
- Nimm nicht an, dass [] Range Checks durchführt
 - Nutze at(), wenn Range Checks garantiert werden müssen
- Elemente werden in Container kopiert
 - Deep Copy für die Hauptobjekte
 - Shallow Copy, wenn die Hauptobjekte Pointer auf andere Objekte sind
- Traue bei Performanz nicht deiner Intuition, sondern führe Messungen durch
- `std::unordered_map` ist die C++ Umsetzung einer Hash Tabelle
- Übergebe Container als Referenz und return sie als Wert/Kopie
- Nutze die ()-Initialisierung, um die Größe mit anzugeben, und die { }-Initialisierung für Sequenzen von Elementen